

Performance Impact and Mitigation Approaches of Latency in ROS-based Autonomous Systems

Thomas Kramp*, Jan Steckel*[†]

*Faculty of Applied Engineering: Department of Electronics-ICT - University of Antwerp, Antwerp, Belgium,

[†]Flanders Make Strategic Research Centre, Lommel, Belgium

Abstract—The continuous research and development in the field of autonomous vehicles (AVs) is accelerating the transition from human driven vehicles towards a fully driverless option. Thus, it necessitates a reliable performance of time-critical applications. A firmware often used during the development of such systems is the Robot Operating System (ROS). However, it is a well studied and documented fact that ROS has inherent latency issues. To solve these real-time processing issues, the Robot Operating System 2 (ROS2) was developed, improving upon scheduling and real-time applications, and adding build-in security, yet some latency issues still persist. This paper aims to investigate the impact of these latency issues on the performance of AV systems and explore potential mitigation strategies if needed.

Index Terms—Autonomous vehicles, ROS, ROS2

I. INTRODUCTION

Automated driving technology has been a topic of wide research due to its potential for smart urban mobility, though an autonomous vehicle (AV) can only be referred to as truly autonomous if it is able to perform all driving functions independently without external intervention [?]. However, legislators and policy makers are still unprepared to implement regulations for true AVs, not only due to practical concerns, but also because of ethical and legal implications [?]. When an AV is in use, it relies on a multitude of sensors to perceive its environment, such as cameras, LIDAR and RADAR. Yet, when an accident occurs, it is often unclear who is to blame: the manufacturer of the vehicle, the software developer or the vehicle's driver. This is why control must always be in the hand of a human driver.

Even though fully AVs may never be completely implemented, there is still a significant amount of potential in Driver Assistance Systems (DAS) and the research towards it.

Master's thesis to obtain the diploma Master of Science in: Electronics and Information and Communication Technology. Academic year 2024-2025. This Master's thesis is an examination document that may not have been corrected for any errors in its final version. Reproduction and copying of this publication or parts thereof is prohibited without the prior written consent of both the supervisor(s) and the author(s). For requests or information regarding the publishing and/or use and/or realisation of parts of this publication, please contact the university at which the author is registered. Prior written consent of the supervisor(s) is also required for the use of the (original) methods, products, circuits and programmes described in this thesis for industrial or commercial use and for the submission of this publication for participation in scientific prizes or competitions.

This document has been assessed by supervisor(s) and/or supervisors in accordance with the faculty Master's thesis regulations.

Level	Description
1	Most functions are controlled by the driver
2	At least one driver assisted function is automated
3	Driver is able to shift safety-critical functions to the system
4	Fully autonomous in most driving scenarios
5	Full automation that is equal to a human driver

TABLE I
LEVELS OF VEHICLE AUTONOMY [?], [?].

According to the Federal Automated Vehicles Policy, a vehicle can still be defined as an AV if the driver is able to shift safety-critical functions to the system [?] (level 3 autonomy or higher in Table ??). Included in these safety-critical system, which were developed during the research towards AVs, are Anti-lock Braking Systems (ABS), Adaptive Cruise Control (ACC) and Lane Departure Warning (LDW) [?], [?]. However, for the driver to be able to hand over control, the system needs to support a wide variety of sensors to detect the surroundings reliably and make real-time decisions based on that data. The Robot Operating System (ROS) is widely used for these purposes [?].

ROS is capable of handling programs written in various languages and running across multiple machines, in which each sensor and actuator is represented as a node that consists of a C++ program or Python script that would perform a specific task. Since the framework is open-source, various packages have been developed by a multitude of institutions and developers, making it simple and quick to implement a wide range of tasks [?]. However, ROS functions as a middleware layer rather than a standalone Operating System (OS), thus the scheduling of tasks are subject to both the ROS and underlying OS schedulers, making for complex timing behavior [?]. Even so, both ROS and ROS2 are widely used for AV development, leading to the possibility that their real-time complications are either acceptable or negligible within an AV context.

II. BACKGROUND

A. ROS and ROS2 latency

ROS was initially designed as a platform to build single-robot applications on. It supports a network of processes, called nodes, that each perform a specific task [?].

Fig. 1. ROS and ROS2 Stack [?].

These are connected by a central node, named “the ROS master node”, that communicates with each node and acts as a liaison between them [?]. Introducing a single point of failure and a bottleneck [?].

ROS utilizes three communication patterns for all node interactions [?], [?]:

- Topics, which act as a Publisher-Subscriber (Pub-Sub) architecture, enabling publishers to broadcast a topic over a channel to which any subscriber can listen.
- Services, which act as a synchronous client/server model, in which a client node sends a request to a server node and waits for it to respond, unable to perform another task while waiting.
- Actions, which act as an asynchronous client/server model, in which a client node sends a request to a server node, except the client node can execute other task while waiting for the response of the server node.

These patterns utilize ROSTCP and ROSUDP (as seen in Figure ??), which are based on the TCP and UDP protocols, respectfully [?]. Leading to added latency in regards to the TCP overhead—due to acknowledgements, retransmissions and ordered delivery—and UDP not guaranteeing delivery.

ROS2 transitions from custom protocols to the Data Distribution Service (DDS) middleware for its communication infrastructure (as seen in Figure ??), with it being designed for high-performance, real-time and interoperable Pub-Sub communication [?]. Integrating the DDS removes the need for a master node, removing the bottleneck and single point of failure; and adds Quality of Service (QoS) policies to the communication patterns, giving more control over message delivery in case of Topics, and improving reliability of both Services and Actions [?]. However, some added features impede its real-time performance, such as:

- The build-in security capabilities that, if used, will increase the latency drastically [?].
- The default callback executors lacking a priority mechanism, causing buffer overflows and missed deadlines [?].

B. ROS latency for AVs

An essential measurement for an AV is the time needed to come to a complete halt. The stopping time therefore consist of the (1) reaction time, (2) the time needed to press the breaks, (3) the start to decelerate and (4) the time to stop [?]. Only (1) and (2) are crucial regarding communication latency within an AV, since the system needs to act as follows within that time frame:

- 1) Sensor takes a measurement.
- 2) Measurement is transferred to the OS.
- 3) OS makes a decision.
- 4) Command is transferred to an actuator.

5) Actuator executes its task.

Dedicated studies show that this time, in regards to human reaction, averages around 0.3s [?], thus the latency on the internal communication must stay below that value.

Today's smart cars—from manufacturers such as Toyota, Porsche, Volkswagen, BMW and Tesla—utilize real-time intra-vehicle wired connection protocols to transmit sensor data and actuator commands, with their data rates ranging between 20Kb/s and 20Mb/s [?]. Table ?? shows the performance of 5 different systems that use ROS2 within an End-to-End (E2E) Pub-Sub setup. These tests produced data rates comparable to the ones used in current cars, though the latency alone of system (3) equals the reaction time as stated above, making it unsuitable for AV applications.

Notably, the three systems with the lowest latency (those being (1), (2) and (5)) do not have the highest specifications. These systems do explicitly implement Intra-Process Communication (IPC), by which messages are sent via in-process memory instead of the ROS2 middleware layer. Thus, the latency produced by ROS2 is less dependent on the hardware and more dependent on how it is implemented. This is best shown in [?], where they tested different DDSs and modes (as seen in Table ??).

However, it should be noted that systems (3) and (4) do not explicitly state that they do or do not use IPC, and that they process LiDAR data, which can produce more than 10MB of data per packet. In addition, [?] tested single- and multi-threaded processing, the latter also being tested with loaned messages¹ and IPC, finding that IPC achieved the lowest latency at the cost of being highly unstable when not implemented correctly.

C. ROS latency due to security

As previously mentioned, the internal encryption of ROS2 can increase the latency drastically, doubling it in wired applications and tripling the latency in wireless applications [?]. Using a VPN (SSL/STL) is a better alternative, since the latency only increases with 25% and the throughput remains unchanged [?], compared to halved with the internal encryption, though encryption is redundant for an AV internally, since any data generated by the sensors is used in real-time and then discarded. Connections with sensors and actuators should be wired to reduce latency. Only the wireless connections need to have encryption integrated.

D. ROS2 plug-in frameworks to reduce latency

1) *Autoware*: Autoware is a software based on ROS2 that is designed for AVs which are intended to drive in urban areas. Similar to ROS, Autoware has a wide support of external modules—such as sensors and actuators—yet its focus

¹With loaned messages both the publisher and subscriber utilize the same data buffers.

Index	CPU	Clock speed	RAM	Latency	Packet size	Citation	E2E connection
1-1	Intel Core i5 3470	3.2GHz	16GB	Packet deadline at 100ms	64KB - 4MB	[?]	With 1-2
1-2	Intel Core i5 2320	3.0GHz	8GB	The same as 1-1	The same as 1-1	[?]	With 1-1
2	Intel i7-6800K	3.4GHz	32GB	Communications were below 1ms	8MB	[?]	None mentioned
3	Intel Xeon E2278 GE	3.3GHz	32GB	Average latency of 300ms	None mentioned	[?]	With LiDAR
4	AMD EPYC 7313P	3.0GHz	32GB	Average latency of 152ms	None mentioned	[?]	With LiDAR
5	Quad-Core Cortex-A72	1.5GHz	Max. 8GB	4.5ms	500KB	[?]	With RaspBerry Pi 4B

TABLE II
THE SYSTEM SPECIFICATION AND PACKET SIZES USED TO TEST ROS2 LATENCY.

Implementation	Maintained the packet deadline	For packet size
FastRTPS DDS	Could not maintain	256B
Connex DDS	Reliably maintains	64KB
OpenSlice DDS	Under best-effort maintains	1MB
IPC	Maintains	4MB

TABLE III
PARAPHRASED VERSION OF "TABLE 4: CAPABILITIES OF ROS1 AND/OR ROS2 FOR EACH DATA TRANSPORT" IN [?].

Fig. 2. A package configuration diagram of Autoware [?].

lies with LiDAR and camera technology. To achieve real-time processing, it filters and pre-processes the point cloud data that is generated by the LiDAR sensor [?]. The result is that a group of raw data points, within a certain cubic area, is reduced to a singular centroid, subsequently downsampling the point cloud. The resulting point cloud is utilized in both its localization and detection, the latter influencing its predictions in how the surrounding objects will be moving compared to the car itself. Using both its location and prediction it will decide on the next action to perform (as can be seen in Figure ??).

The resulting E2E execution time (equaling all four time stamps mentioned in section ??) would average around 50ms for an Intel Core i7-6700K (4.0GHz) using 32GB of memory [?]. This is a 6th of the latency with the comparable system using ROS2 (as seen with example (3) in Table ??).

2) *PAAM*: In ROS2 any callback can directly call the hardware accelerators². When multiple processes complete their callback at the same time and try to utilize those accelerators, there is no priority on who will acquire the accelerators first, resulting in missed deadlines and latency. PAAM is a Priority-driven Accelerator Access Management framework built on top of ROS2. It offers accelerator access as a service, such that any ROS2 callback needs to send a request to the PAAM server—a standalone executor dedicated to accelerator access—also making the scheduling more predictable [?]. This reduces the latency on high priority tasks and will drastically increase the latency on non-critical tasks (as can be seen in Figure ??).

²The CPU is a processor capable of generic computations, while an accelerator is an add-on that helps the CPU with specific tasks such as the GPU with graphical computations and the TPU with machine learning computations.

Fig. 3. A E2E latency comparison?? between PAAM, PiCAS?? and ROS2 [?].

3) *ROSGM*: Since ROS2 only utilizes a singular fixed management policy for all GPU tasks, its resources aren't used efficiently. The ROS2 GPU Management (ROSGM) framework tries to solve this by providing a platform where one can add different policies, in case of "applications that need long time throughput" or "tasks that require real-time execution", allowing for the application to switch between modes, such as "heavy-load" mode or "urgent" mode for the previous examples respectfully. To test this framework an Intel i5-9400H CPU processor⁵ using 24GB of memory was used. Their results show that, in applications with an increasing amount of callbacks, this framework reduces latency with 10-20ms for to ROS2 systems with latencies of between 70-90ms [?].

III. CONCLUSION

This paper asks if the latency issues inherent to ROS make it unsuitable for AV applications, since the communication (which includes latency) must be quicker than human reaction to be of any use. System (3) and arguably (4) (as seen in Table ??) have a latency that make the performance exceed this reaction time, despite these being high Performance systems with the highest specifications listed in this paper. Showing that ROS is unsuitable for AV applications if it is not implemented with latency reduction in mind.

In order to ensure ROS is applicable, changing settings such as bypassing the scheduler via IPC—as shown with systems (1), (2) and (5) (as seen in Table ??)—and not utilizing security are shown to reduce the latency to such a degree that it can be used in AVs. In addition, the integration of different frameworks compatible with ROS have been discussed, showing that pre-processing data (in case of Autoware) or changing how the scheduler performs (in case of PAAM and ROSGM) can have a certain impact on its performance. Thus, leading to

³Chains 1-6 represent critical chains, with chain 1 having the highest priority going down to chain 6 having the lowest priority, while BE 1-2 represent non-critical best-effort chains [?].

⁴Priority-driven Chain-Aware Scheduling (PiCAS) is an older (in comparison to PAAM) scheduling framework that works on top of the ROS2 stack. [?].

⁵The used CPU clock speed was not mentioned, even so the Intel website states it can reach 4.3GHz [?].

the conclusion that ROS can be used for AV applications if applied correctly.